JavaScript Array Methods — Complete Reference

A practical reference covering all array methods grouped by purpose, with simple examples.

---

1. Mutator Methods (Modify the Original Array)

`push()`

Adds one or more elements to the **end** of the array. Returns the new length.

```javascript
const arr = [1, 2, 3];
arr.push(4);          // returns 4
console.log(arr);     // [1, 2, 3, 4]
```

`pop()`

Removes the **last** element. Returns the removed element.

```javascript
const arr = [1, 2, 3];
const removed = arr.pop();  // 3
console.log(arr);           // [1, 2]
```

`unshift()`

Adds one or more elements to the **beginning**. Returns the new length.

```javascript
const arr = [2, 3];
arr.unshift(1);       // returns 3
console.log(arr);     // [1, 2, 3]
```

`shift()`

Removes the **first** element. Returns the removed element.

```javascript
const arr = [1, 2, 3];
const first = arr.shift();   // 1
console.log(arr);            // [2, 3]
```

`splice()`

Adds, removes, or replaces elements at any position. Returns removed elements.

```javascript
const arr = [1, 2, 3, 4];
arr.splice(1, 2, 'a', 'b');  // removes 2 elements at index 1, inserts 'a','b'
console.log(arr);             // [1, 'a', 'b', 4]
```

`sort()`

Sorts the array in place. Default is lexicographic (string) order — pass a compare function for numbers.

```javascript
const nums = [10, 2, 30, 4];
nums.sort();                        // [10, 2, 30, 4] sorted as strings
nums.sort((a, b) => a - b);         // [2, 4, 10, 30] numeric ascending
```

`reverse()`

Reverses the array in place.

```javascript
const arr = [1, 2, 3];
arr.reverse();        // [3, 2, 1]
```

`fill()`

Fills all or part of the array with a static value.

```javascript
const arr = [1, 2, 3, 4];
arr.fill(0);          // [0, 0, 0, 0]
arr.fill(9, 1, 3);    // fill index 1 to 3 (exclusive) with 9
```

`copyWithin()`

Copies part of the array to another location within the same array.

```javascript
const arr = [1, 2, 3, 4, 5];
arr.copyWithin(0, 3);  // [4, 5, 3, 4, 5]
```

2. Accessor Methods (Return New Data, Don't Modify)

`concat()`

Merges arrays and returns a new array.

```javascript
const a = [1, 2];
const b = [3, 4];
const result = a.concat(b);   // [1, 2, 3, 4]
```

`slice()`

Returns a shallow copy of a portion. Original is untouched.

```javascript
const arr = [1, 2, 3, 4, 5];
arr.slice(1, 3);      // [2, 3]
arr.slice(-2);        // [4, 5]
```

`indexOf()`

Returns the first index of a value, or `-1` if not found.

```javascript
const arr = ['a', 'b', 'c', 'b'];
arr.indexOf('b');     // 1
arr.indexOf('z');     // -1
```

`lastIndexOf()`

Returns the last index of a value, or `-1` if not found.

```javascript
const arr = ['a', 'b', 'c', 'b'];
arr.lastIndexOf('b');  // 3
```

`includes()`

Checks if a value exists. Returns `true` / `false`.

```javascript
```

```javascript
const arr = [1, 2, 3];
arr.includes(2);      // true
arr.includes(5);      // false
```

`join()`

Joins all elements into a string with a separator.

```javascript
const arr = ['Naveen', 'Automation', 'Labs'];
arr.join(' ');        // "Naveen Automation Labs"
arr.join('-');        // "Naveen-Automation-Labs"
```

`toString()`

Converts array to a comma-separated string.

```javascript
const arr = [1, 2, 3];
arr.toString();       // "1,2,3"
```

`at()`

Returns the element at a given index. Supports negative indexes (counts from end).

```javascript
const arr = [10, 20, 30, 40];
arr.at(1);            // 20
arr.at(-1);           // 40 (last element)
```

---

3. Iteration Methods (Functional, Don't Modify)

`forEach()`

Executes a function for each element. Returns `undefined`.

```javascript
```

```javascript
const arr = [1, 2, 3];
arr.forEach((num, i) => console.log(i, num));
// 0 1
// 1 2
// 2 3
```

`map()`

Transforms each element and returns a new array.

```javascript
const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2);   // [2, 4, 6]
```

`filter()`

Returns a new array containing only elements that match a condition.

```javascript
const nums = [1, 2, 3, 4, 5];
const evens = nums.filter(n => n % 2 === 0);  // [2, 4]
```

`reduce()`

Reduces the array to a single value, processing left to right.

```javascript
const nums = [1, 2, 3, 4];
const sum = nums.reduce((acc, curr) => acc + curr, 0);  // 10
```

`reduceRight()`

Same as `reduce()`, but processes right to left.

```javascript
const arr = [[1, 2], [3, 4], [5, 6]];
arr.reduceRight((acc, curr) => acc.concat(curr), []);   // [5, 6, 3, 4, 1, 2]
```

`find()`

Returns the **first** element that matches a condition, or `undefined`.

```javascript
```

```javascript
const users = [{ id: 1, name: 'A' }, { id: 2, name: 'B' }];
users.find(u => u.id === 2);   // { id: 2, name: 'B' }
```

`findIndex()`

Returns the index of the first matching element, or `-1`.

```javascript
const nums = [10, 20, 30];
nums.findIndex(n => n > 15);   // 1
```

`findLast()` *(ES2023)*

Returns the **last** matching element.

```javascript
const nums = [1, 2, 3, 4];
nums.findLast(n => n % 2 === 0);   // 4
```

`findLastIndex()` *(ES2023)*

Returns the index of the last matching element.

```javascript
const nums = [1, 2, 3, 4];
nums.findLastIndex(n => n % 2 === 0);   // 3
```

`some()`

Returns `true` if **at least one** element matches the condition.

```javascript
const nums = [1, 2, 3];
nums.some(n => n > 2);   // true
```

`every()`

Returns `true` if **all** elements match the condition.

```javascript
const nums = [2, 4, 6];
nums.every(n => n % 2 === 0);   // true
```

`flat()`

Flattens nested arrays. Default depth is 1.

```javascript
const arr = [1, [2, 3], [4, [5, 6]]];
arr.flat();           // [1, 2, 3, 4, [5, 6]]
arr.flat(2);          // [1, 2, 3, 4, 5, 6]
arr.flat(Infinity);   // fully flatten
```

`flatMap()`

Maps each element, then flattens the result by one level.

```javascript
const arr = [1, 2, 3];
arr.flatMap(n => [n, n * 2]);   // [1, 2, 2, 4, 3, 6]
```

`entries()`

Returns an iterator of `[index, value]` pairs.

```javascript
const arr = ['a', 'b', 'c'];
for (const [i, v] of arr.entries()) {
  console.log(i, v);
}
// 0 'a'
// 1 'b'
// 2 'c'
```

`keys()`

Returns an iterator of indices.

```javascript
const arr = ['a', 'b', 'c'];
[...arr.keys()];      // [0, 1, 2]
```

`values()`

Returns an iterator of values.

```javascript
```

```javascript
const arr = ['a', 'b', 'c'];
[...arr.values()];    // ['a', 'b', 'c']
```

---

4. Static Methods (Called on `Array` itself)

`Array.isArray()`

Checks if a value is an array.

```javascript
Array.isArray([1, 2, 3]);     // true
Array.isArray("hello");        // false
```

`Array.from()`

Creates an array from an iterable or array-like object.

```javascript
Array.from("hello");                       // ['h','e','l','l','o']
Array.from({ length: 3 }, (_, i) => i);    // [0, 1, 2]
```

`Array.of()`

Creates an array from arguments (unlike `Array()` which behaves oddly with one number).

```javascript
Array.of(7);          // [7]
Array(7);             // empty array of length 7 (gotcha!)
Array.of(1, 2, 3);    // [1, 2, 3]
```

---

5. Immutable Methods *(ES2023 — Return New Array)*

These are the non-mutating versions of `sort`, `reverse`, `splice`, and index assignment.

`toSorted()`

Returns a sorted copy. Original stays unchanged.

```javascript
```

```
const arr = [3, 1, 2];
const sorted = arr.toSorted();   // [1, 2, 3]
console.log(arr);                 // [3, 1, 2] — unchanged
```

`toReversed()`

Returns a reversed copy.

```javascript
const arr = [1, 2, 3];
const rev = arr.toReversed();    // [3, 2, 1]
console.log(arr);                 // [1, 2, 3] — unchanged
```

`toSpliced()`

Returns a spliced copy.

```javascript
const arr = [1, 2, 3, 4];
const out = arr.toSpliced(1, 2, 'a');   // [1, 'a', 4]
console.log(arr);                       // [1, 2, 3, 4] — unchanged
```

`with()`

Returns a new array with the element at the given index replaced.

```javascript
const arr = [1, 2, 3];
const out = arr.with(1, 99);     // [1, 99, 3]
console.log(arr);                 // [1, 2, 3] — unchanged
```

Quick Decision Cheat Sheet

Goal	Use
Add/remove from end	`push` / `pop`
Add/remove from start	`unshift` / `shift`
Add/remove anywhere	`splice`
Transform every element	`map`

Goal	Use
Pick matching elements	`filter`
Sum/aggregate to one value	`reduce`
Find one element	`find` / `findIndex`
Check any/all match	`some` / `every`
Check value exists	`includes`
Flatten nested arrays	`flat` / `flatMap`
Non-mutating sort/reverse	`toSorted` / `toReversed`
Build array from iterable	`Array.from`

Key Notes for SDET Work

- **Prefer immutable methods** (`map`, `filter`, `toSorted`) in test code — they avoid side-effect bugs across test runs.
- `forEach` cannot be broken out of early — use `for...of` or `some` when you need early exit.
- `sort()` mutates and sorts as strings by default — always pass a comparator for numbers.
- `Array.from({ length: N })` is the cleanest way to generate test data arrays.